High-Performance Big Data Analytics Using R

NYC Yellow Taxi Trip Records · January – June 2024 · 19,650,216 cleaned trips

Laboratory Assignment 8 · R Programming · Swaroop Naik (github.com/Swaroop192005/R-Prog-Project)

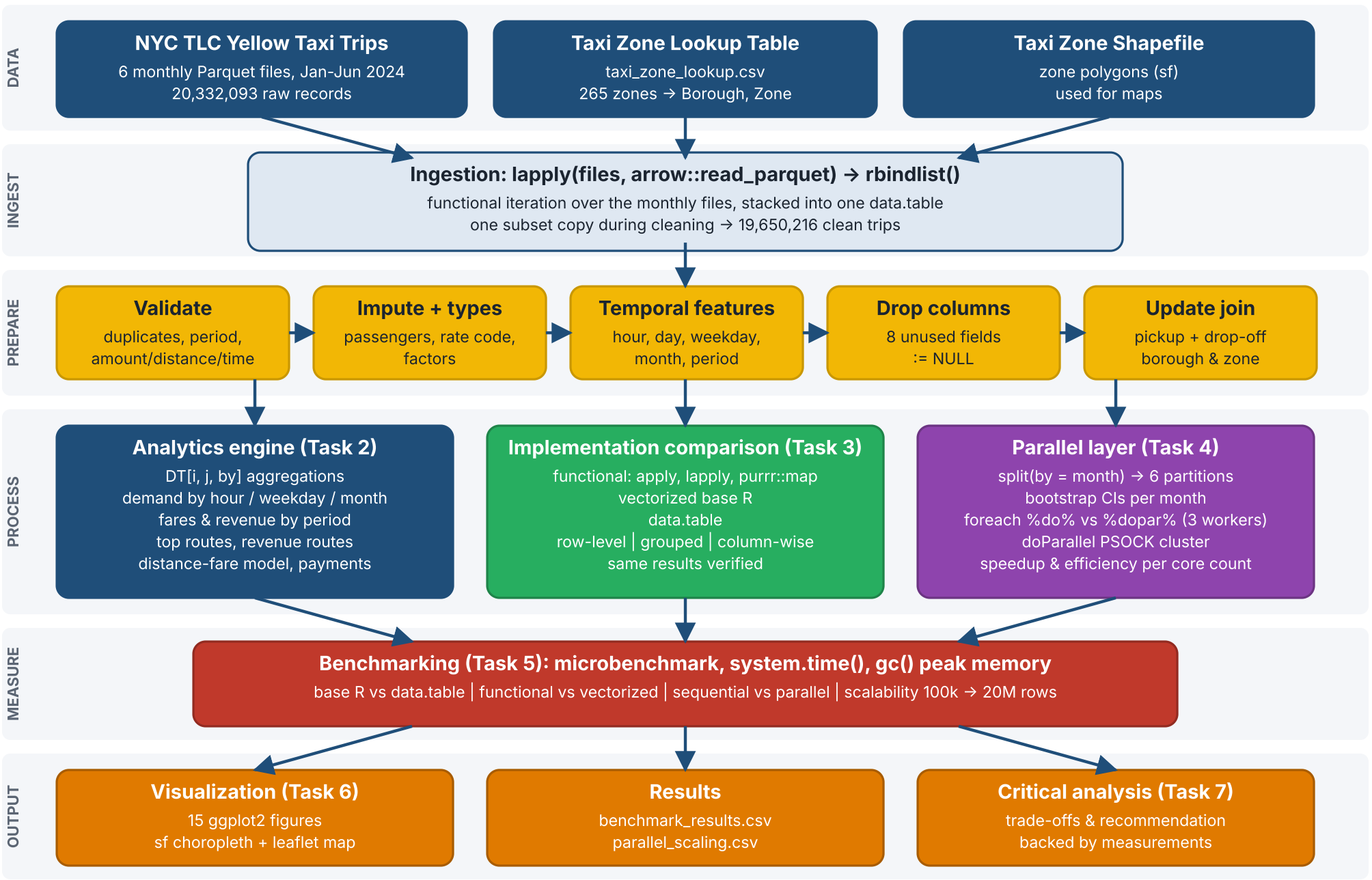
PROBLEM

Millions of taxi-trip records make conventional row-by-row R code slow and memory hungry. Goal: a scalable R workflow that finds demand patterns, high-revenue routes, fare–distance behaviour and payment preferences, and that measures which R programming approach is fastest and most memory efficient.

TECH STACK

- **arrow** – Parquet import
- **data.table** – clean, join, aggregate
- **purrr / apply family** – functional style
- **foreach + doParallel** – parallel
- **microbenchmark**, system.time, gc()
- **ggplot2, sf, leaflet** – visualisation

SYSTEM ARCHITECTURE



18:00 peak pickup hour

76.2% trips paid by credit card

0.936 distance – fare correlation

2.11× parallel speedup (3 cores)

BENCHMARK RESULTS (MEDIAN TIME)

Operation	Fastest	time	Slowest	time	gap
row-level fare per mile	vectorized	0.5 ms	apply()	1.6 s	**3382×**
grouped mean by borough x hour	data.table by	49 ms	sapply() filter per group	13.6 s	**281×**
column means (4 columns)	purrr::map_dbl()	21 ms	apply(, 2)	93 ms	**4×**
filter long expensive trips	data.table DT[cond]	196 ms	base R subset()	576 ms	**3×**
revenue by month x hour	data.table by	386 ms	base R aggregate()	15.1 s	**39×**
sort by total amount	data.table DT[order()]	1.9 s	base R order()	7.0 s	**4×**
join 2M trips with zones	base R match()	32 ms	base R merge()	4.9 s	**157×**
bootstrap per month (sequential vs parallel)	Parallel foreach %dopar% (3 cores)	25.9 s	Sequential lapply()	54.8 s	**2×**

KEY INSIGHTS

- Demand peaks at **18:00**, is lowest at 4–5 AM; **Thu** and **May** are the busiest weekday and month.
- Airport routes dominate revenue: top route **JFK Airport → Outside of NYC**; airport trips are ~10% of trips but ~27% of revenue.
- Fare is almost linear in distance (r = 0.936), but fare per mile falls sharply for longer trips.
- Row-wise **apply()** was **3382×** slower than vectorized code; **merge()** was **30×** slower than a data.table join.
- Parallel bootstrap: **2.11x** speedup on 3 cores; a light task ran at **0.01x** – parallel is not always faster.

RECOMMENDATION

data.table as the core engine, **vectorized** element-wise logic, **functional** iteration over a few large units (files, columns, months), and **foreach/doParallel** only for CPU-heavy independent work, where the benchmark shows the speedup outweighs the start-up and data-transfer overhead.

Data: NYC Taxi & Limousine Commission Trip Record Data and Taxi Zone Lookup

Measured on a 4-core Linux machine, R 4.6.1